

Balanceador de Carga con NGINX y Pruebas con Artillery

Laura Victoria Gallo Payana 2220903

Santiago Duque Valencia 2215270

Kelvin Pablo Martínez Botina 2185430

Nicolas Alberto Cuellar Castellon 2220906

Contexto del problema

El problema central es la falta de distribución eficiente del tráfico, donde un servidor sobrecargado se convierte en un punto único de falla, comprometiendo la continuidad del servicio, algo crítico en entornos empresariales o institucionales.

Además, la creciente necesidad de escalabilidad en servicios en línea exige soluciones que permitan agregar o retirar recursos de forma dinámica sin interrumpir la operación. Los balanceadores de carga emergen como componentes esenciales en infraestructuras modernas, actuando como intermediarios inteligentes que distribuyen las solicitudes entrantes entre múltiples servidores, optimizando el uso de recursos. cómo se optimiza el rendimiento de servicios web, garantizando su disponibilidad y escalabilidad ante cargas variables de usuarios.

Contexto del problema

PROBLEMA: ALTA DEMANDA EN SERVICIOS WEB

Sobrecarga de servidores únicos:

- Alto tráfico simultáneo causa latencia, caídas del servicio y falta de disponibilidad.
- Punto único de falla = riesgo para la continuidad del servicio.

Falta de distribución eficiente:

- Tráfico no distribuido = recursos desbalanceados.
- Escalabilidad limitada sin afectar la operación.

SOLUCIÓN: BALANCEADOR DE CARGA CON NGINX

Distribución inteligente:

- Reparte peticiones entre múltiples servidores.
- Diferentes algoritmos de balanceo (Round Robin, Least Connections, IP Hash).

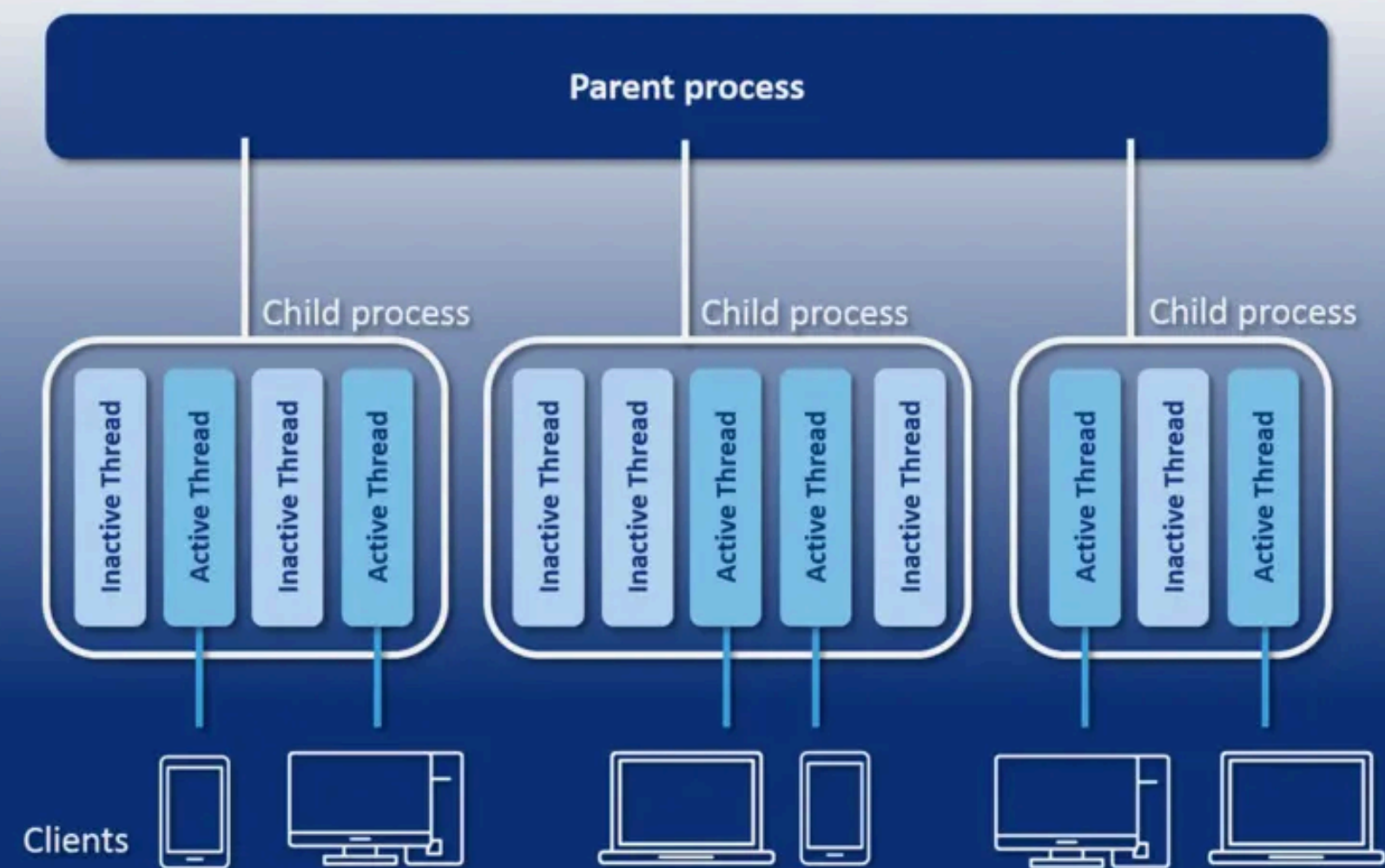
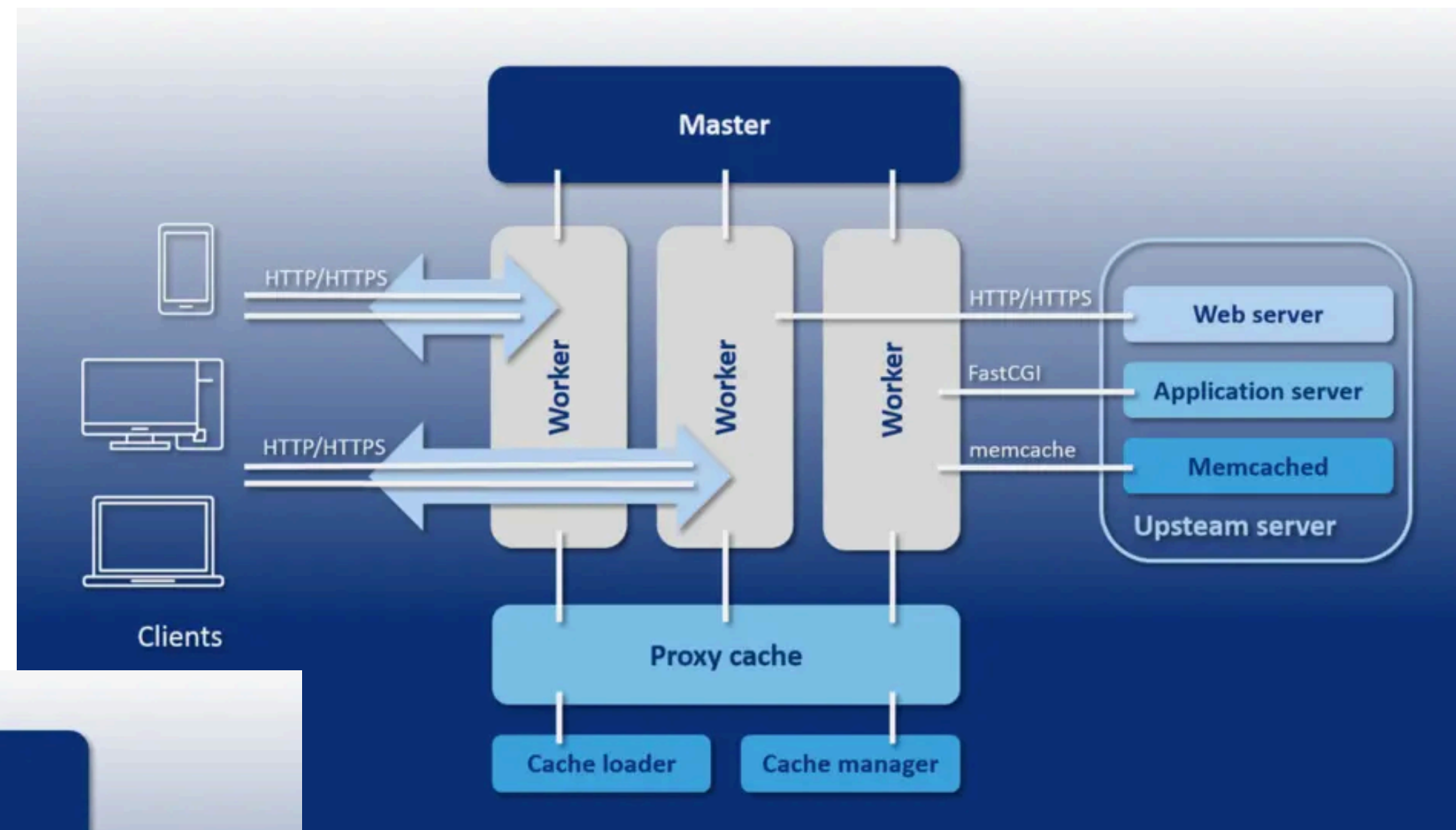
Beneficios:

- Mayor disponibilidad y tolerancia a fallos.
- Escalabilidad flexible y eficiente.
- Configuración avanzada y económica.

Alternativas de solución

Se evaluaron distintas alternativas para implementar el balanceo de carga, considerando herramientas como Apache HTTP Server con mod_proxy_balancer y HAProxy. Sin embargo, **se eligió NGINX** por su eficiencia, bajo consumo de recursos y facilidad de configuración, además de su capacidad para implementar diversos algoritmos de balanceo como Round Robin, Least Connections e IP Hash. Esta flexibilidad permitió adaptar el comportamiento del balanceador a distintos escenarios de carga y requisitos de persistencia de sesión.

Alternativas de solución



Diseño de la solución

El diseño consistió en una arquitectura distribuida con tres máquinas virtuales: un balanceador NGINX y dos servidores web Apache. Se estableció una red privada (192.168.50.0/24) para la comunicación entre los nodos. El balanceador se configuró inicialmente con el algoritmo Round Robin, pero también se probaron otros métodos como IP Hash para mantener la afinidad de sesión y Least Connections para distribución dinámica según la carga activa. Esta estructura permitió evaluar el rendimiento bajo diferentes estrategias de balanceo.

Implementación

La implementación se automatizó mediante Vagrant y VirtualBox, utilizando scripts de aprovisionamiento en Shell para instalar y configurar NGINX y Apache en cada máquina virtual. Se definieron bloques upstream en NGINX con los algoritmos correspondientes (ip_hash, least_conn, y pesos variables para Round Robin). Las pruebas de carga se realizaron con Artillery, simulando desde 3000 hasta 4000 usuarios concurrentes con tasas de solicitudes entre 50 y 400 peticiones por segundo, dependiendo del escenario.

Implementación

ROUND ROBIN

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Crear la configuración del balanceador
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    server 192.168.50.10:80;
    server 192.168.50.20:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

# Activar el sitio y desactivar el por defecto
ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

Reparte equitativamente las peticiones

LEAST CONNECTIONS

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador - Algoritmo: Least Connections"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Configuración con algoritmo least_conn
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    least_conn;
    server 192.168.50.10:80;
    server 192.168.50.20:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/loadbalancer.conf
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

**Envía la nueva petición al servidor que
tenga menos conexiones activas**

Implementación

IP HASH

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador - Algoritmo: IP Hash"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Configuración con algoritmo ip_hash
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    ip_hash;
    server 192.168.50.10:80;
    server 192.168.50.20:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

Selecciona el servidor basándose en la dirección IP de origen

WEIGHTED ROUND ROBIN

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador - Algoritmo: Weighted Round Robin"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Configuración con pesos
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    server 192.168.50.10:80 weight=3;
    server 192.168.50.20:80 weight=1;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/loadbalancer.conf
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

Permite darle más peso (más tráfico) a un servidor más potente.

Implementación

ROUND ROBIN

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Crear la configuración del balanceador
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    server 192.168.50.10:80;
    server 192.168.50.20:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

# Activar el sitio y desactivar el por defecto
ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

Reparte equitativamente las peticiones

LEAST CONNECTIONS

```
#!/usr/bin/env bash
set -e

echo "[INFO] Instalando NGINX como balanceador - Algoritmo: Least Connections"
apt-get update -y
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx

# Configuración con algoritmo least_conn
cat > /etc/nginx/sites-available/loadbalancer.conf <<'EOF'
upstream backend {
    least_conn;
    server 192.168.50.10:80;
    server 192.168.50.20:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

ln -s /etc/nginx/sites-available/loadbalancer.conf /etc/nginx/sites-enabled/loadbalancer.conf
|| true
rm -f /etc/nginx/sites-enabled/default

systemctl enable nginx
systemctl restart nginx
```

**Envía la nueva petición al servidor que
tenga menos conexiones activas**

Pruebas

Se ejecutaron pruebas de carga con Artillery para evaluar el rendimiento de cada algoritmo. Con IP Hash, se simuló una carga media de 3000 usuarios, obteniendo una latencia promedio de 163.2 ms y solo 0.2% de errores. En la prueba de Least Connections con 4000 usuarios, se observaron latencias entre 150-250 ms y picos de hasta 4 segundos. Finalmente, con Weighted Round Robin bajo alta carga, el sistema mostró una degradación progresiva, con latencias máximas de 7 segundos y más del 90% de errores en la fase final.

Discusión de las pruebas

Los resultados confirmaron que IP Hash es ideal para escenarios que requieren persistencia de sesión, con estabilidad y baja tasa de errores. Least Connections demostró ser más eficiente en entornos de carga variable, redistribuyendo dinámicamente las solicitudes, aunque con picos de latencia en momentos de alta concurrencia. Por su parte, Weighted Round Robin mostró limitaciones bajo carga sostenida, saturándose rápidamente cuando los servidores más potentes recibían mayor tráfico. En general, se evidenció la importancia de seleccionar el algoritmo según el tipo de tráfico y la capacidad de los servidores backend.

Conclusiones

La solución desarrollada demostró la importancia de la distribución de tráfico en sistemas web, evidenciando mejoras en la eficiencia, escalabilidad y disponibilidad de los servicios.

Entre los principales logros del proyecto se destacan:

- La verificación exitosa del algoritmo round-robin, que garantizó una distribución equitativa de las solicitudes entre los servidores.
- La medición objetiva del rendimiento a través de Artillery, permitiendo observar métricas reales de latencia y throughput.
- Asimismo, se identificaron posibles líneas de mejora, como la incorporación de monitoreo activo de servidores, la implementación de HTTPS con certificados SSL, y la integración con herramientas de observabilidad como Prometheus y Grafana para la recolección de métricas en tiempo real.

Referencias

1. NGINX, Inc. (2024). NGINX Documentation. Disponible en: <https://nginx.org/en/docs/>
2. HashiCorp. (2024). Vagrant Documentation. Disponible en: <https://developer.hashicorp.com/vagrant/docs>
3. Artillery.io. (2024). Artillery - Modern load testing toolkit. Disponible en: <https://www.artillery.io/docs>
4. Apache Software Foundation. (2024). Apache HTTP Server Documentation. Disponible en: <https://httpd.apache.org/docs/>
5. Stallings, W. (2020). Data and Computer Communications, 11th Edition. Pearson.
6. Tanenbaum, A. S., & Wetherall, D. J. (2017). Computer Networks, 5th Edition. Pearson Education.
7. <https://www.ionos.com/es-us/digitalguide/servidores/know-how/apache-vs-nginx-una-comparativa-de-servidores-web/>

The background is white with a decorative border at the top and bottom. The top border consists of a red section on the right and a blue section on the left. The bottom border consists of a red section on the left and a blue section on the right. Scattered across the white background are numerous rounded squares of various sizes, some in light blue and some in a darker blue. The text "Muchas Gracias" is centered in a bold, dark blue font.

**Muchas
Gracias**